COMPETITIVE GAMEFI INFRASTRUCTURE

GULLIANT

When a character sells, the buyer inherits the asset — not the player.

THE OPERATIONAL PROBLEM

// This is not a design preference. It is a production failure mode.

SCENARIO

"When a high-ranked PvP character is sold, the buyer inherits the seller's matchmaking tier, while the seller loses it. Neither side intended this outcome."

Observed in early conversations with GameFi teams

- → **Inconsistent Truth:** Indexers and analytics pipelines become de facto sources of truth, bypassing on-chain logic.
- → **Mixed Reconstruction:** Derived state (Rank, MMR, progression) is reconstructed from fragmented sources.
- → **State Leaks:** Player-specific data remains tied to wallets after NFT characters are traded.

THIS IS NOT A DESIGN PROBLEM.

The boundary between **Character (Asset)** and **Player (Identity)** is not systemically enforced. It relies on service-layer convention, which fails at scale.

WHERE IT ACTUALLY BREAKS

ON-CHAIN LAYER

Separation exists in code but lacks **Strict Invariants**. Programs allow state updates that should be logically isolated.

INDEXER LAYER

Mixed reconstruction. Indexers treat on-chain events and off-chain service logs as equal inputs, creating **State Drift**.

SERVICE LAYER

Backend services consume derived data as absolute truth. When an NFT is sold, the new owner inherits **"Ghost State"** from the previous player.

Every GameFi protocol that ships tradable characters without this will build their own version of it. Most will get it wrong.

`Gulliant enforces the boundary on-chain.`

Indexer and service layers enforce it through integration discipline.

THE GULLIANT PRIMITIVE

Making the on-chain boundary authoritative.

- → Clean Reconstruction: Derived state can be reconstructed exclusively from the user log — eliminating the need for mixed-source reconstruction.
- → Verifiable Transition: Character progression (Rank/Stats) is tied to the asset via a deterministic hash-chain.
- → Atomic Migration: Player state (Permissions/Limits) migrates between wallets via single-use authorization.

Without Gulliant:

→ conflicting state updates can be accepted
→ user history becomes inconsistent

With Gulliant:

→ invalid state transitions are rejected on-chain
→ log integrity is preserved

The question is not whether to enforce this boundary.
The question is what it costs to integrate.

ENFORCEMENT REALITY

Gulliant makes the user log the authoritative on-chain source.

Teams that eliminate mixed-source reconstruction and index exclusively from the user log eliminate state drift by construction.

In a complete integration, the user log is the only valid source for derived player state.

INTEGRATION REALITY

WHAT IT REPLACES

- → Manual sync logic between Indexers and DBs
- → Fragile wallet-to-character mapping tables
- → Custom "Admin-only" migration scripts

INTEGRATION EFFORT

- → **Gulliant PDA:** Acts as the Root of Truth for user logs
- → **CPI Interface:** Your game program calls Gulliant to commit state deltas
- → **Network Logic:** Compatible with existing Solana indexers — requires configuring the user log as the canonical source for player-derived state.

Gulliant does **NOT** replace your game logic. It provides the **Boundary Guard** that ensures your game logic remains consistent across trades and wallet rotations.

INTEGRATION PATH

This is deployed. Here is how to verify it.

PROGRAM ID

DyQAWvMytpgRiE2T9bhzSFCe9b9actobfHFB7RVReX7g

IMPLEMENTATION

- → Add CPI calls to Gulliant on player-affecting events.
- → Use Gulliant user log as the canonical source for derived state.
- → Validate state deltas on-chain via hash-chain checks.

NEXT STEP

Devnet pilot around one real gameplay flow.

Target a specific high-friction trade scenario (e.g., Rank migration) to prove system integrity.